OASIS VM 부팅 프로세스 분석

개요

OASIS VM은 Intel TDX(Trust Domain Extensions) 가상 머신을 위한 최소한의 Linux 배포판입니다. 2단계 부팅 구조를 사용하여 보안성과 검증 가능성을 제공합니다.

전체 부팅 아키텍처

```
OVMF 펌웨어 → Stage 1 (initramfs) → Stage 2 (squashfs) → 애플리케이션
```

주요 구성 요소

1. **OVMF 펌웨어**: TDX 지원 UEFI 펌웨어
2. **Stage 1**: 커널 + initramfs (검증 및 Stage 2 로딩)
3. **Stage 2**: 실제 루트 파일시스템 (기본/Podman 버전)

---

1. OVMF 펌웨어 단계

역할

- TDX 가상 머신의 초기 부팅을 담당
- UEFI 환경 제공
- Stage 1 커널 로딩

구현 세부사항

빌드 설정

- **기반**: EDK2 (tianocore) 프로젝트
- **TDX 지원**: IntelTdxX64.dsc 플랫폼 사용
- **보안 부팅**: 선택적 지원 (PACKAGECONFIG)

주요 기능

```
# TDX 지원 빌드
${S}/OvmfPkg/build.sh -a X64 -b RELEASE -t GCC5 -p OvmfPkg/IntelTdx/IntelTdxX64.
```

출력 파일

- `ovmf.tdx.fd` : TDX 지원 펌웨어
- `ovmf.tdx.code.fd` : 코드 영역
- Shell.efi: UEFI 셸

프로세스 흐름

1. TDX 환경 초기화
2. UEFI 서비스 제공
3. Stage 1 커널 로딩 및 실행

---

2. Stage 1 부팅 프로세스

역할

- dm-verity를 통한 Stage 2 무결성 검증
- 파티션 설정 및 스토리지 준비
- Stage 2로의 루트 전환

구현 세부사항

커널 설정

```
# 주요 커널 기능
KERNEL_FEATURES += "features/scsi/disk.scc"
KERNEL_FEATURES += "cfg/virtio.scc cfg/paravirt_kvm.scc cfg/fs/ext4.scc"
KERNEL_FEATURES += "tdx.scc security-mitigations.scc disk-encryption.scc"
```

initramfs 구성

- **크기 제한**: 20MB (INITRAMFS_MAXSIZE)
- **필수 도구**: busybox, cryptsetup, e2fsprogs, util-linux
- **초기화 스크립트**: `/init`

Stage 1 초기화 프로세스 (/init)

1. 가상 파일시스템 마운트

```
mkdir -p /proc /sys /dev /dev/pts
mount none -t proc /proc
mount none -t sysfs /sys
mount none -t devtmpfs /dev
```

```
mount none -t devpts /dev/pts
```

2. 커널 명령줄 파싱

Stage 2 설정을 커널 명령줄에서 추출:

```
# 필수 매개변수
oasis.stage2.roothash=<HASH> # dm-verity 루트 해시
oasis.stage2.hash_offset=<OFFSET> # 해시 오프셋

# 선택적 매개변수
oasis.stage2.storage_mode=<MODE> # 스토리지 모드 (disk/ram/custom)
oasis.stage2.storage_offset=<OFFSET> # 스토리지 파티션 오프셋
oasis.stage2.storage_size=<SIZE> # 스토리지 파티션 크기
```

3. 파티션 설정 (dm-linear)

```
# 루트 파티션만 (스토리지 없음)
echo "0 ${stage2_size} linear /dev/vda 0" | dmsetup create part-rootfs

# 루트 + 스토리지 파티션
echo "0 ${stage2_storage_offset} linear /dev/vda 0" | dmsetup create part-roo
echo "0 ${stage2_storage_size} linear /dev/vda ${stage2_storage_offset}" | dmse
```

4. dm-verity 설정

```
veritysetup open \
--hash-offset=${stage2_hash_offset} \
--panic-on-corruption \
/dev/mapper/part-rootfs \
stage2 \
/dev/mapper/part-rootfs \
${stage2_root_hash}
```

5. Stage 2 마운트

```
mkdir -p /stage2
mount -t squashfs /dev/mapper/stage2 /stage2
```

6. 스토리지 설정

스토리지 모드에 따른 처리:

디스크 모드 (disk):

```
# 임시 암호화 키 생성
storage_key=$(dd if=/dev/urandom bs=1024 count=1 2>/dev/null | tr -dc 'a-zA


# LUKS2 포맷
echo -n "${storage_key}" | cryptsetup luksFormat \
--type luks2 \
```

```
--integrity hmac-sha256 \
--disable-locks \
--header /tmp/storage-luks2-header \
/dev/mapper/part-storage

# 암호화 볼륨 열기
echo -n "${storage_key}" | cryptsetup open \
--type luks2 \
--disable-locks \
--header /tmp/storage-luks2-header \
/dev/mapper/part-storage \
storage

# 파일시스템 생성 및 마운트
mkfs.ext4 /dev/mapper/storage
mount /dev/mapper/storage /stage2/storage
```

RAM 모드 (ram):

```
mount none -t tmpfs -o size=${stage2_storage_size} /stage2/storage
```

7. 루트 전환

```
exec switch_root /stage2 /init
```

보안 특징

- **측정된 부팅**: 커널 명령줄이 RTMR2에 측정됨
- **무결성 검증**: dm-verity를 통한 Stage 2 검증
- **임시 암호화**: 디스크 스토리지의 임시 암호화

---

3. Stage 2 부팅 프로세스

역할

- 실제 애플리케이션 실행 환경 제공
- 네트워크 설정
- 컨테이너 런타임 (Podman 버전)

구현 세부사항

파일시스템 구조

```
/stage2/
├── init # Stage 2 초기화 스크립트
├── etc/oasis/
| ├── init.post-registration
| └── post-registration.d/
| ├── 00-network # 네트워크 설정
| └── 80-podman # Podman 설정 (Podman 버전만)
```

```
├── storage/ # 읽기-쓰기 스토리지
├── var -> /storage/var # 변수 데이터
└── ...
```

Stage 2 초기화 프로세스

1. 기본 설정 (00-network)

```
# 루프백 인터페이스 활성화
ip link set lo up

# tmpfs 마운트 (resolv.conf용)
mount none -t tmpfs /tmp

# DHCP 클라이언트 시작
udhcpc -i eth0 -q -n
```

2. Podman 설정 (80-podman, Podman 버전만)

```
# cgroups v2 마운트
mount -t cgroup2 none /sys/fs/cgroup

# 공유 메모리 설정
mkdir -p /dev/shm
mount -t tmpfs none /dev/shm
```

```
# 컨테이너 디렉토리 정리 및 생성
rm -rf /storage/containers/run /storage/containers/net /var/lib/cni
mkdir -p /storage/containers/{run,graph,graph/tmp,net}

# TUN 디바이스 권한 설정
chmod 0666 /dev/net/tun

# Podman 시스템 마이그레이션 및 정리
podman system migrate
podman system prune --external

# 컨테이너 시작 (정의된 경우)
if [ -d "/etc/oasis/containers" ]; then
cd /etc/oasis/containers
podman-compose up --detach --remove-orphans --force-recreate &
fi
```

Stage 2 버전별 차이점

기본 버전 (stage2-basic)

- **패키지**: stage2-base, busybox, libgcc, libssl, ca-certificates
- **용도**: 기본적인 런타임 환경

- **크기**: 최소화

Podman 버전 (stage2-podman)

- **추가 패키지**: podman, podman-compose, python3-unixadmin
- **용도**: 컨테이너 기반 애플리케이션
- **기능**: 컨테이너 오케스트레이션

스토리지 관리

- **읽기 전용**: Stage 2 루트 파일시스템 (squashfs)
- **읽기-쓰기**: `/storage` 디렉토리
- **변수 데이터**: `/var` → `/storage/var` 바인드 마운트
- **임시 데이터**: `/run`, `/tmp` (tmpfs)

---

4. 빌드 프로세스

Yocto/OpenEmbedded 기반 빌드

빌드 환경

```
# Docker 컨테이너 사용
DOCKER run --rm -it \
-v $(pwd):/workdir \
-w /workdir \
crops/poky@sha256:fe6976cc917b6e1d6d11583ef628a16a165c91e76715608e4a2ce4c8433661
```

```
make build-inner
```

빌드 대상

```
source poky/oe-init-build-env build/
bitbake oasis-vm-stage1 # Stage 1 이미지
bitbake oasis-vm-stage2-basic # 기본 Stage 2
bitbake oasis-vm-stage2-podman # Podman Stage 2
bitbake ovmf # OVMF 펌웨어
```

출력 아티팩트

```
build/artifacts/
├── ovmf.tdx.fd # TDX 펌웨어
├── stage1.bin # Stage 1 커널+initramfs
├── stage2-basic.tar.bz2 # 기본 Stage 2 템플릿
└── stage2-podman.tar.bz2 # Podman Stage 2 템플릿
```

메타 레이어 구조

```
meta-oasis-vm/
├── conf/ # 배포 및 머신 설정
├── recipes-containers/ # 컨테이너 관련 패키지
├── recipes-core/ # 핵심 이미지 및 패키지
```

```
├── recipes-devtools/ # 개발 도구

└── recipes-kernel/ # 커널 설정
```

---

5. 보안 및 검증

신뢰 체인

1. **OVMF**: TDX 측정 및 검증
2. **Stage 1**: 커널 명령줄 측정 (RTMR2)
3. **dm-verity**: Stage 2 무결성 검증
4. **암호화**: 임시 스토리지 암호화

측정 지점

- **RTMR0**: OVMF 펌웨어
- **RTMR1**: Stage 1 커널 및 initramfs
- **RTMR2**: 커널 명령줄 (Stage 2 설정 포함)
- **RTMR3**: Stage 2 루트 해시

보안 기능

- **읽기 전용 루트**: Stage 2는 squashfs로 변조 불가
- **임시 암호화**: 스토리지 데이터 보호
- **네트워크 격리**: 최소한의 네트워크 설정
- **컨테이너 격리**: Podman을 통한 애플리케이션 격리

---

6. 사용 사례

ROFL 애플리케이션

- Oasis CLI의 `rofl build` 명령과 함께 사용
- Stage 2 템플릿을 기반으로 애플리케이션 이미지 생성

개발 및 테스트

```
# 빌드 환경 진입
make shell

# 개별 컴포넌트 빌드
bitbake oasis-vm-stage1 -c compile
```

이 문서는 OASIS VM의 전체 부팅 프로세스와 각 단계별 구현 세부사항을 다룹니다. 2단계 부팅 구조를 통해 보안성과 검증 가능성을 제공하며, TDX 환경에서 안전한 애플리케이션 실행을 가능하게 합니다.